

Constructed demonstration. All numerical records are teaching fixtures and must not be presented as real research evidence.

Early Window Records with Bounded Late-Data Replacement

Abstract

A collector that emits each window only once must choose when to stop waiting for late packets. We change this emission policy: R5 issues a complete record after 5 s, then replaces it as late samples arrive until 120 s after window closure. Revision ordering prevents delayed aggregate deliveries from restoring an older result. In the constructed bursty replay of 48 windows, R5 improves from 31 exact records initially to 48 finally, matching the final agreement of the 120 s one-shot collector F120; the 5 s one-shot I5 remains at 31. R5 retains 124 KiB, compared with 96 KiB for F120 and 12 KiB for I5, and generates 17 corrections. Recovery remains bounded: R5 and F120 finish with 44 exact windows in `beyond_retention`. These records illustrate earlier provisional output with later repair, not a measured execution-speed gain.

1 Introduction

An event-time collector can report a window before all its packets arrive. The previous collector waits and emits once: a longer wait admits later packets, whereas a shorter wait leaves the emitted result unchanged even when missing packets subsequently arrive. We instead allow an early record to be replaced while its window state is retained. This change creates a receiver-side obligation: a corrected aggregate must survive repeated and out-of-order deliveries without reverting to an older result or mixing two versions. R5 therefore couples bounded retention with complete, revision-ordered record replacement. The work changes emission and replacement in an existing collector; it does not introduce event-time windows, aggregation, deduplication, transactions or version numbers. Three arrival profiles and one diagnostic examine what the complete change enables, what it costs and where it stops repairing records.

2 Implementation

The inherited collector partitions packets into 20 s windows by event timestamp and computes count and sum. Each packet has a stable (`stream_id`, `sample_id`); the collector retains seen sample IDs with the window and updates its aggregate only for a newly seen ID. A duplicate input packet is ignored, independently of any duplicate aggregate delivery to the receiver. Every emitted record contains five fields: `stream_id`, `window_end`, `revision`, `count` and `sum`. The receiver displays the mean in Eq. (1).

$$\bar{x}_w = S_w / N_w \quad (1)$$

F120 waits 120 s after window closure and emits once; I5 waits 5 s and emits once without correction. R5 also first emits after 5 s, but retains the sample-ID set and aggregate until 120 s after closure. A newly accepted late packet that changes the aggregate increments revision and generates another complete record. Retention ends at this fixed deadline, independently of arrival time; expiry removes the aggregate and sample-ID set. Packets for an expired window are logged and rejected, never moved to the next window. All variants use the same event-time closure rule, and the same expiry rule applies across profiles.

Whole-record replacement avoids adding duplicated corrections twice. For each window, the receiver accepts only a revision greater than the stored one, rejecting repeated and stale deliveries. A transaction prevents mixing count and sum from different versions. The early delta approach had the duplication flaw but no measured results. U5 retains R5's record generation and transactional pair replacement, removing only revision ordering. The prototype has one writer per stream; reconnects, concurrent writers, storage

failures and long-running deployment were not tested. Complete exactly-once processing is not established.

3 Replay and results

Each of `on_time`, `bursty` and `beyond_retention` contains 48 distinct windows. One replay was constructed per listed variant/profile; these are not repeated benchmark trials, and there are no timing confidence intervals or error bars. Only `bursty` was used for U5. Table 1 retains every supplied row. First-output wait is a configured setting, not compute latency. Initial and final exact agreement require both count and sum to match the complete offline packet list, with final agreement checked after replay. Peak retained state measures serialized aggregates and sample IDs, not total process memory. Generated corrections are revisions after first emission, excluding duplicate transport transmissions.

Table 1. Constructed replay records. Exact windows are initial/final out of 48; state is retained-state KiB.

Profile	Variant	Wait s	Exact initial/final	Corrections	State KiB
<code>on_time</code>	F120	120	48/48	0	96
<code>on_time</code>	I5	5	48/48	0	12
<code>on_time</code>	R5	5	48/48	0	124
<code>bursty</code>	F120	120	48/48	0	96
<code>bursty</code>	I5	5	31/31	0	12
<code>bursty</code>	R5	5	31/48	17	124
<code>bursty</code>	U5	5	31/38	17	124
<code>beyond_retention</code>	F120	120	44/44	0	96
<code>beyond_retention</code>	I5	5	36/36	0	12
<code>beyond_retention</code>	R5	5	36/44	8	124

The `bursty` comparison separates early output from eventual agreement. I5 and R5 both start with 31 of 48 exact windows. R5 generates 17 corrections and finishes at 48, whereas I5 remains at 31; F120 is exact in all 48 at its first emission. All needed packets arrive by 90 s. U5 generates the same corrections but finishes at 38 because older deliveries overwrite newer records in ten windows, demonstrating the role of revision ordering in this trace. Beyond retention, twelve windows are incomplete at 5 s: eight are completed before 120 s, producing eight R5 corrections, but four needed packets arrive at 180 s. R5 and F120 therefore finish at 44 exact windows; I5 remains at 36. In `on_time`, all variants are exact throughout and no corrections are generated. R5 retains 124 KiB in every profile, exceeding F120's 96 KiB and I5's 12 KiB; U5 matches R5's generation-side state and correction counts. Thus early repair adds retained state and record transmissions, gives no agreement benefit in `on_time`, and cannot repair arrivals after eviction. No other parameter selection or post-profile retuning was evaluated.

4 Conclusion

Bounded, revision-ordered replacement lets the constructed collector emit at 5 s and later reach F120's final agreement within the supplied retention limit. The `bursty` diagnostic shows why complete replacement still needs revision ordering; transactional pair updates keep each displayed mean tied to one accepted record. This behavior costs retained state and extra transmissions, cannot repair evicted windows, and does not establish production robustness or general superiority over stream-processing systems.

Sources and scope

The implementation notebook and `results.csv` supplied with this demonstration are the only sources. There is no literature review or external novelty claim. All supplied rows, including negative outcomes, remain part of the manuscript.